

თბილისის თავისუფალი უნივერსიტეტი
მათემატიკის, კომპიუტერული მეცნიერების და ინჟინერიის
სკოლა (MACS[E])
კომპიუტერული მეცნიერებისა და მათემატიკის პროგრამა

არევაძე მათე
ჭიჭინაძე ქეთი
გელაშვილი გიორგი

საბაკალავრო პროექტი
„Peared“

ხელმძღვანელი: აბსანძე ლუკა

თბილისი
2026

ანოტაცია

ერთობლივი მუშაობა ერთსა და იმავე ტექსტურ ფაილზე დღეს ყოველდღიურობის ნაწილია - იქნება ეს წყვილში პროგრამირება, სემინარი, ლაბორატორიული სამუშაო თუ ტექნიკური გასაუბრება. ამის მიუხედავად, არსებული ინსტრუმენტების დიდი ნაწილი ორ დაშვებას ეყრდნობა: დოკუმენტი უნდა „აიტვირთოს“ რომელიმე კომპანიის ცენტრალურ სერვერზე (მაგ. Google Docs) და ყველა მონაწილეს სტაბილური ინტერნეტი უნდა ჰქონდეს. ჩვენი გამოცდილებით, უნივერსიტეტის აუდიტორიაში ორივე დაშვება ხშირად ირღვევა: ეკრანის გაზიარება ხშირად არ მუშაობს და სემინარზე ყველა სტუდენტს პროფესორის ლეპტოპის გარშემო შეკრება უწევს.

ამ პრობლემის გადასაჭრელად შევექმენით Peered, კროსპლატფორმული desktop აპლიკაცია ერთობლივი ტექსტური რედაქტირებისთვის, რომელიც მუშაობს როგორც ლოკალურად, ისე გლობალურად. სესიის წამომწყები (ჰოსტი) საკუთარ კომპიუტერზე უშვებს მსუბუქ relay სერვერს, რომელსაც აპლიკაცია ავტომატურად მართავს, ხოლო დანარჩენი მონაწილეები უბრალოდ ბმულით უერთდებიან. დოკუმენტი არასდროს ტოვებს მონაწილეთა კომპიუტერებს.

პარალელური რედაქტირებისას ცვლილებების უკონფლიქტოდ შერწყმას უზრუნველყოფს CRDT (Conflict-free Replicated Data Type) ალგორითმი, კონკრეტულად, YATA ვარიანტი, რომელიც კოლაბორაციული ედიტინგის ყველაზე ცნობილი ბიბლიოთეკის, Yjs-ის, საფუძველია. თითოეული კლიენტი დოკუმენტის საკუთარ ასლს ფლობს და ოპერაციები ისეა აგებული, რომ ნებისმიერი თანმიმდევრობით მიღების შემთხვევაში ყველა მონაწილე ერთსა და იმავე ტექსტს დაინახავს. სისტემა სამი დამოუკიდებელი კომპონენტისგან შედგება: Rust-ზე დაწერილი CRDT ბიბლიოთეკა, Tauri Framework-ზე აგებული desktop კლიენტი (React/Monaco ფრონტენდით) და Go-ზე დაწერილი WebSocket relay სერვერი. გლობალური წვდომისთვის აპლიკაცია ავტომატურად ხსნის Cloudflare Tunnel-ს, რაც პორტების გახსნისა და ქსელის კონფიგურაციის გარეშე აძლევს დამოუკიდებელ მონაწილეებს შეერთების საშუალებას.

შედეგად მივიღეთ მომუშავე პროდუქტი, რომლითაც რამდენიმე ადამიანს ერთდროულად შეუძლია ერთი და იმავე ფაილის რედაქტირება და უფლებების მართვა, და რომელიც ნელი ინტერნეტის პირობებშიც კი ეფექტურად მუშაობს.

Abstract

Working together on the same text file is part of everyday life today - whether it is pair programming, a seminar, a lab session or a technical interview. Despite this, most existing tools rest on two assumptions: the document must be uploaded to some company's central server (e.g. Google Docs), and every participant must have a stable internet connection. In our experience, both assumptions frequently break down in a university classroom: screen sharing often fails, and the whole seminar ends up crowding around the professor's laptop.

To solve this problem we built Peared, a cross-platform desktop application for collaborative text editing that works both locally and globally. The session initiator (the host) runs a lightweight relay server on their own machine, managed automatically by the application, while the other participants simply join via a link. The document never leaves the participants' computers.

Conflict-free merging of concurrent edits is guaranteed by a CRDT (Conflict-free Replicated Data Type) algorithm, specifically the YATA variant that underlies Yjs, the best-known collaborative editing library. Every client owns its own replica of the document, and operations are constructed so that, regardless of the order in which they are received, all participants will see the same text. The system consists of three independent components: a CRDT library written in Rust, a desktop client built on the Tauri framework (with a React/Monaco frontend), and a WebSocket relay server written in Go. For global access the application automatically opens a Cloudflare Tunnel, which lets remote participants connect without port forwarding or any network configuration.

The result is a working product with which several people can edit the same file simultaneously and manage write permissions, and which performs well even under a slow internet connection.

სარჩევი

ანოტაცია	ii
Abstract	iii
ცხრილების, დიაგრამების, ფორმულების და სურათების ჩამონათვალი	v
შესავალი	1
ძირითადი ნაწილი	3
პროექტის თემასთან დაკავშირებული სფეროს კვლევა	3
პროექტის ტექნიკური მხარე	5
მიღებული შედეგი და შეფასება	19
დასკვნა	23
გამოყენებული ლიტერატურა	24
დანართები	25

ცხრილების, დიაგრამების, ფორმულების და სურათების ჩამონათვალი

დიაგრამები

- 1 სისტემის საერთო არქიტექტურა: ჰოსტის კომპიუტერზე
გაშვებული გეითვეი და მასთან დაკავშირებული კლიენტები . . . 6
- 2 კლიენტის (Tauri აპლიკაციის) შრეები და ქვესისტემები 8
- 3 CRDT დოკუმენტის სტრუქტურა: ბლოკების linked list (ქვემოთ,
tombstone წითლადაა) და მასზე აგებული პოზიციური B+
ინდექსი (ზემოთ, კვანძებში, ხილული სიმბოლოების ჯამები) . 10
- 4 ბლოკის ჩასმა ნაბიჯ-ნაბიჯ: ბლოკი ერთდროულად ემატება
დაკავშირებულ სიას და ხის ფოთოლს, შემდეგ კი ხილული
სიგრძის დელტა ფესვამდე ვრცელდება 11
- 5 ბლოკის წაშლა ნაბიჯ-ნაბიჯ: ბლოკი სიაში რჩება tombstone-
ად, ხილული სიგრძე ნულდება და უარყოფითი დელტა ხის
ჯამებს ასწორებს 12

სურათები

- 1 ფაილების სექცია: შენახვა, გახსნა, Fork და ბოლო ფაილების
სია 20
- 2 ჰოსტის ხედი: gateway/tunnel სტატუსი, მოსაწვევი ბმულები და
Invite დილაკი (statusline-ზე ჩანს HOST როლი და room id) 20
- 3 სტუმრის ხედი read-only რეჟიმში და peers პანელი, სადაც ჩანს
ჰოსტი და read-only სტუმარი 21
- 4 მეტრიკების პანელი: გეითვეის live სტატისტიკა (კლიენტები,
ოთახები, გადაგზავნილი ფრეიმები, uptime) 21

ცხრილები

- 1 პოზიციის ძეგნის დრო ინდექსამდე და ინდექსის შემდეგ 17

შესავალი

პროექტის თემის შერჩევა საკმაოდ პრაქტიკული დაკვირვებით დაიწყო. სწავლის პერიოდში არაერთხელ აღმოვჩენილვართ სიტუაციაში, როდესაც სემინარზე ან ლაბორატორიულ სამუშაოზე საჭირო იყო, რომ რამდენიმე სტუდენტს, ან სტუდენტსა და ლექტორს, ერთდროულად ემუშავა ერთსა და იმავე კოდზე. სტანდარტული გამოსავალი, ეკრანის გაზიარება - უნივერსიტეტის ქსელში ხშირად უბრალოდ არ მუშაობდა: ინტერნეტი წყდებოდა, ვიდეო ნაკადი ჭედავდა და საბოლოოდ მთელი ჯგუფი ლექტორის ლეპტოპის გარშემო იკრიბებოდა, რომ სემინარში მონაწილეობა მიეღო. Google Docs-ის ტიპის ინსტრუმენტები კი, გარდა იმისა, რომ კოდის რედაქტირებისთვის მოუხერხებელია, სტაბილურ ინტერნეტსა და ცენტრალურ სერვერს საჭიროებს, ანუ ზუსტად იმას, რაც აუდიტორიაში ხშირად არ გვქონდა.

აქედან გაჩნდა კითხვა: შესაძლებელია თუ არა ისეთი ერთობლივი რედაქტორის შექმნა, რომელიც ცენტრალურ სერვისზე საერთოდ არ იქნება დამოკიდებული და იმუშავებს როგორც ლოკალურ ქსელში (თუნდაც ინტერნეტის გარეშე), ისე ინტერნეტით დაშორებულ მონაწილეებთანაც და ექნება ownership სისტემა, რომელიც ლექტორს მისცემს საშუალებას, აკონტროლოს სესია და მართოს სტუდენტთა პრივილეგიები. თემის ამ მიმართულებით განვითარებაში დაგვეხმარა ჩვენი ხელმძღვანელი, ლუკა აბსანძე, რომელმაც პროექტის საწყის ეტაპზე რამდენიმე საინტერესო იდეა და მიმართულება შემოგვთავაზა, საიდანაც საბოლოო არჩევანი გავაკეთეთ.

პრობლემა, რომელსაც პროექტი ეხმიანება, შემდეგნაირად შეიძლება ჩამოყალიბდეს: რეალურ დროში ერთობლივი ტექსტური რედაქტირების არსებული გადაწყვეტილებები თითქმის ყველა შემთხვევაში მოითხოვს (1) დოკუმენტის მესამე მხარის სერვერზე შენახვას, (2) მუდმივ ინტერნეტკავშირს და (3) ხშირად ანგარიშების შექმნასა და კონკრეტულ ეკოსისტემაზე მიბმას. ეს სამივე მოთხოვნა პრობლემურია მაშინ, როდესაც სამუშაო ფაილი კონფიდენციალურია, ქსელი არასტაბილურია ან უბრალოდ გვინდა, რომ ერთი ოთახის ფარგლებში, ზედმეტი ინფრასტრუქტურის გარეშე ვითანამშრომლოთ.

პროექტის მიზანი იყო სრულფასოვანი, კროსპლატფორმული desktop აპლიკაციის შექმნა, რომელიც:

- საშუალებას აძლევს ერთ მომხმარებელს (ჰოსტს), წამოიწყოს სესია და დანარჩენებს, უბრალოდ ბმულით შეუერთდნენ მას;
- პარალელურ რედაქტირებას უკონფლიქტოდ წყვეტს CRDT

ალგორითმით, ისე, რომ ყველა მონაწილე გარანტირებულად ერთსა და იმავე ტექსტს ხედავდეს;

- მუშაობს როგორც ლოკალურ ქსელში ინტერნეტის გარეშე, ისე გლობალურად - Cloudflare Tunnel-ის მეშვეობით, ქსელის ყოველგვარი კონფიგურაციის გარეშე;
- ჰოსტს აძლევს სესიის სრულ კონტროლს: სტუმრების ჩაწერის უფლებების მართვას, სესიის დაწყება/დასრულებას და ფაილების ლოკალურად შენახვას.

პროექტის ფარგლები შეგნებულად შემოვსაზღვრეთ: სესია ერთ დოკუმენტზე მუშაობს, დოკუმენტი უბრალო ტექსტია (კოდი, კონფიგურაცია, ჩანაწერები) და არა ფორმატირებული rich text, ხოლო სესია ეფემერულია, ის ჰოსტის აპლიკაციასთან ერთად იბადება და კვდება, თუმცა ნამუშევრის ფაილად შენახვა ნებისმიერ მომენტშია შესაძლებელი.

მეთოდოლოგიის მხრივ სისტემა სამ დამოუკიდებლად აწყობად კომპონენტად დავყავით: CRDT ბიბლიოთეკა Rust-ზე (crdt-core), desktop კლიენტი Tauri ფრეიმვორკზე (Rust ბექენდი + React/TypeScript ფრონტენდი Monaco რედაქტორით) და მსუბუქი relay სერვერი Go-ზე (gateway).

პოტენციური მომხმარებლები, პირველ რიგში, სტუდენტები და ლექტორები არიან - სემინარები, ლაბორატორიული სამუშაოები, ჯგუფური პროექტები, თუმცა იგივე სცენარები ბუნებრივად ვრცელდება წყვილში პროგრამირებაზე, ტექნიკურ გასაუბრებებზე, მენტორინგსა და ნებისმიერ სიტუაციაზე, სადაც რამდენიმე ადამიანს ერთი ტექსტის ერთდროული რედაქტირება სჭირდება ცენტრალური სერვისისთვის მონაცემების გადაცემის გარეშე.

და ბოლოს, პროექტს აქვს რამდენიმე მოსალოდნელი შეზღუდვა. სესია ჰოსტის კომპიუტერზეა მიბმული: თუ ჰოსტი გაითიშა, სესია სრულდება (თუმცა თითოეულ მონაწილეს დოკუმენტის სრული ასლი რჩება). გლობალური წვდომა დამოკიდებულია Cloudflare-ის უფასო trycloudflare.com სერვისის ხელმისაწვდომობაზე - მისი გაუმართაობის შემთხვევაში აპლიკაცია ლოკალური ქსელის რეჟიმში აგრძელებს მუშაობას.

ძირითადი ნაწილი

პროექტის თემასთან დაკავშირებული სფეროს კვლევა

Collaborative editing და არსებული ტექნოლოგიები

რეალურ დროში ერთობლივი რედაქტირება დღეს ყველასთვის ნაცნობი ფუნქციაა - Google Docs-მა ის მასობრივ პროდუქტად აქცია, მაგრამ ეს არ არის ერთადერთი ტექნოლოგია ამ სფეროში. თუ ამ ინსტრუმენტებს დავაკვირდებით, თითქმის ყველა მათგანი ერთსა და იმავე არქიტექტურულ ნიმუშს მიჰყვება: არსებობს ცენტრალური სერვერი, რომელიც დოკუმენტის „კანონიკურ“ ვერსიას ფლობს, ხოლო კლიენტები ამ სერვერს უკავშირდებიან. ეს მიდგომა კარგად მუშაობს კომერციული SaaS პროდუქტისთვის, მაგრამ მასთან ერთად მოდის დამოკიდებულებები: მუდმივი ინტერნეტი, მესამე მხარის ინფრასტრუქტურა და დოკუმენტის მონაცემების cloud storage-ში შენახვა.

ტექნიკური თვალსაზრისით, პარალელური რედაქტირებისას conflict-resolution-ის ორი ტექნიკა არსებობს. პირველი არის **Operational Transformation (OT)** [3]: როდესაც ორი მომხმარებელი ერთდროულად ასწორებს ტექსტს, მათი ოპერაციები (ჩასმა/წაშლა პოზიციებით) ერთმანეთის მიმართ „ტრანსფორმირდება“ ისე, რომ განსხვავებული თანმიმდევრობით შესრულების შემდეგაც ერთი და იგივე შედეგი მივიღოთ. OT გამოიყენება Google Docs-სა და Etherpad-ში, თუმცა მას ორი ცნობილი სისუსტე აქვს: ტრანსფორმაციის ფუნქციების კორექტულად დაწერა ძალიან რთულია და პრაქტიკული OT სისტემები თითქმის ყოველთვის ცენტრალურ სერვერს საჭიროებს, რომელიც ოპერაციებს ერთიან თანმიმდევრობას მიანიჭებს.

მეორე მიდგომაა **CRDT, Conflict-free Replicated Data Type** [2]. აქ იდეა შებრუნებულია: იმის ნაცვლად, რომ ოპერაციები ერთმანეთს მოვარგოთ, თავად მონაცემთა სტრუქტურა ისეა აგებული, რომ ოპერაციები კომუტაციური იყოს, ანუ მიღების თანმიმდევრობას შედეგი არ შეეცვალოს. თითოეული ჩასმული სიმბოლო (ან სიმბოლოთა ბლოკი) იღებს გლობალურად უნიკალურ იდენტიფიკატორს და პოზიციის ნაცვლად სწორედ ამ იდენტიფიკატორებით აღიწერება „სად“ ჩაჯდა ტექსტი. ამის ფასი ისაა, რომ წაშლილი ელემენტების კვალი, ე.წ. tombstone-ები, გარკვეული ხნით უნდა შევინახოთ, სამაგიეროდ ცენტრალური კოორდინატორი საერთოდ აღარ არის საჭირო: CRDT ბუნებრივად ერგება peer-to-peer და დეცენტრალიზებულ არქიტექტურებს.

ტექსტისთვის განკუთვნილი არაერთი CRDT ალგორითმი არსებობს (WOOT, RGA, Logoot/LSEQ და სხვა), მაგრამ პრაქტიკაში ყველაზე

წარმატებული აღმოჩნდა **YATA, Yet Another Transformation Approach** [1], რომელიც Yjs ბიბლიოთეკის [4] საფუძველია. Yjs დღეს ალბათ ყველაზე ფართოდ გამოყენებადი open source ერთობლივი რედაქტირების ძრავაა და სწორედ მისმა პრაქტიკულმა წარმატებამ დაგვარწმუნა, რომ YATA სწორი არჩევანია. YATA-ში ყოველი ჩასმა იმასსოვრებს არა მხოლოდ საკუთარ იდენტიფიკატორს, არამედ „წარმოშობის“ მეზობლებსაც (origin left/right), თუ რომელ ორ ელემენტს შორის დაიბადა ის. როდესაც ორი მონაწილე ერთსა და იმავე ადგილას ერთდროულად ჩასვამს ტექსტს, კონფლიქტი დეტერმინისტული წესით წყდება და, რაც მნიშვნელოვანია, პარალელურად აკრეფილი სიტყვები ერთმანეთში არ „აირევა“.

რატომ CRDT და რატომ YATA

ჩვენი არჩევანი სამ არგუმენტს ეყრდნობოდა. ჯერ ერთი, პროექტის მთავარი მოტივაცია სწორედ ცენტრალურ სერვისზე დამოკიდებულების მოხსნა იყო, OT-ის სერვერცენტრული ბუნება ამ მიზანს პირდაპირ ეწინააღმდეგება, CRDT კი საშუალებას გვაძლევს, სერვერული კომპონენტი უბრალო გადამგზავნ „მილად“ ვაქციოთ, რომელსაც დოკუმენტის შინაარსი საერთოდ არ ესმის. მეორე, CRDT-ის კორექტულობის მსჯელობა ლოკალურია: საკმარისია დავამტკიცოთ, რომ ოპერაციების ინტეგრაცია კომუტაციურია, და ქსელის ნებისმიერი გაჭიანურება თუ თანმიმდევრობის დარღვევა უსაფრთხო ხდება. მესამე, YATA-ს აქვს რეალურ პროდუქტში (Yjs) წლების განმავლობაში გამოცდილი დიზაინი, საიდანაც ბევრი პრაქტიკული ხერხი გადმოვიღეთ: სიმბოლოების ნაცვლად ბლოკებით (რამდენიმე სიმბოლოს გაბმული მონაკვეთებით) ოპერირება და ბლოკის გაყოფა (splitting) შუაში ჩასმისას, წაშლების კომპაქტური, დიაპაზონებად შენახვა და ა.შ. დამატებით შთაგონებას ვიღებდით diamond-types პროექტიდანაც [5], რომელმაც გვიჩვენა, რომ CRDT დოკუმენტს პოზიციური ინდექსით მნიშვნელოვნად აჩქარება შეუძლია.

რატომ Cloudflare Tunnel

დეცენტრალიზებული მიდგომის ერთი მოუხერხებელი დეტალი ისაა, რომ ჰოსტის კომპიუტერი ინტერნეტიდან პირდაპირ მიუწვდომელია: სახლისა თუ უნივერსიტეტის ქსელებში NAT და firewall გარე შეერთებებს ბლოკავს, ხოლო პორტების ხელით გახსნა (port forwarding) რიგითი მომხმარებლისთვის შეუძლებელი მოთხოვნაა. ამ პრობლემას ვჭრით Cloudflare Tunnel-ით [8]: ჰოსტის აპლიკაცია cloudflared პროგრამას უშვებს, რომელიც ჰოსტიდან Cloudflare-ის ქსელისკენ გამავალ

კავშირს ამყარებს და სანაცვლოდ დროებით საჯარო მისამართს (*.trycloudflare.com) იღებს. გამავალი კავშირი NAT-ს უპრობლემოდ გადის, მომხმარებელს არაფრის კონფიგურაცია არ სჭირდება, კავშირი TLS-ითაა დაცული და სერვისი უფასოა. თუ გვირავის გახსნა ვერ მოხერხდა, აპლიკაცია ამაზე არ ვარდება, სესია ლოკალური ქსელის ბმულით გრძელდება.

პროექტის ბუნება, მოთხოვნები და კომერციალიზაცია

პროექტი არსებულ სფეროში არსებული გამოწვევის, ცენტრალიზებულ ინფრასტრუქტურაზე დამოკიდებულების, ალტერნატიულ გზას გვთავაზობს. ჩვენ არ გამოგვიგონია ახალი თანხმობის ალგორითმი; სიახლე კომბინაციაშია: local-first desktop აპლიკაცია, რომელიც სესიის ინფრასტრუქტურას (relay სერვერს და public tunnel-ს) მომხმარებლისვე კომპიუტერზე, ავტომატურად და შეუმჩნევლად მართავს. ცალკე აღნიშვნის ღირსია ჩვენი garbage collection სისტემა, რომელიც, CRDT იმპლემენტაციებში გავრცელებული მიდგომებისგან განსხვავებულ, Host-Authoritative პროტოკოლს იყენებს (დეტალურად - ოპტიმიზაციების თავში).

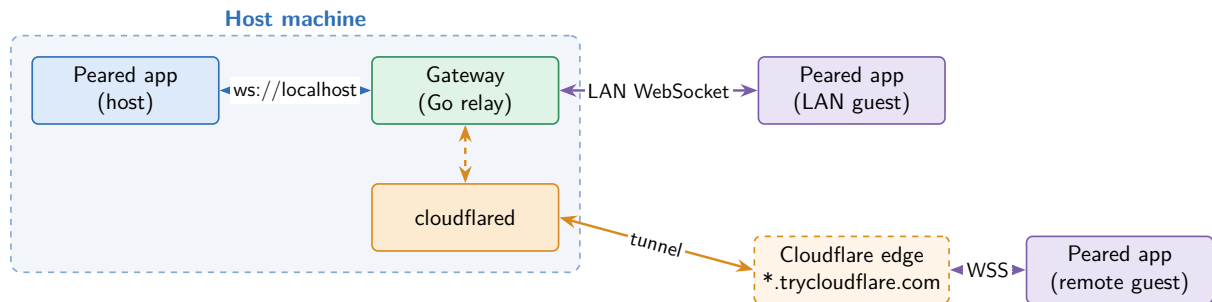
პროექტის ტექნიკური მხარე

სისტემის საერთო არქიტექტურა

Peared-ის სისტემა სამი გაშვებადი (runtime) კომპონენტისგან შედგება:

1. **desktop აპლიკაცია** (tauri-app), ეს ნაწილი ეშვება ყველა მონაწილის მიერ. სწორედ აქ ცხოვრობს დოკუმენტის CRDT ასლი, Monaco რედაქტორი და სესიის მართვის მთელი ლოგიკა.
2. **გეითვეი** (gateway), Go-ზე დაწერილი WebSocket relay სერვერი, რომელსაც მხოლოდ ჰოსტის აპლიკაცია უშვებს დამხმარე (side-car) პროცესად სესიის დაწყებისას და კლავს სესიის დასრულებისას. ის მდგომარეობას თითქმის არ ინახავს, უბრალოდ გადააგზავნის ბაიტებს ოთახის წევრებს შორის.
3. **Cloudflared გვირავი**, არასავალდებულო დამხმარე პროცესი, რომელიც ჰოსტის გეითვეის დროებით საჯარო *.trycloudflare.com მისამართზე აქვეყნებს, რათა ლოკალური ქსელის გარეთ მყოფმა სტუმრებმაც შეძლონ შეერთება.

ქსელური ტოპოლოგია hub-and-spoke მოდელს მიჰყვება: ყველა მონაწილე (თავად ჰოსტის კლიენტიც კი) ერთსა და იმავე გეითვეის უკავშირდება WebSocket-ით, ხოლო გეითვეი მიღებულ ყოველ ფრეიმს დანარჩენებს გადაუგზავნის. ამით ვიღებთ peer-to-peer სისტემის თვისებებს (დოკუმენტი მხოლოდ მონაწილეებთანაა, ცენტრალური სერვისი არ არსებობს) სრული mesh-ქსელის სირთულის გარეშე - თითოეულ კლიენტს მხოლოდ ერთი კავშირი სჭირდება.



დიაგრამა 1. სისტემის საერთო არქიტექტურა: ჰოსტის კომპიუტერზე გაშვებული გეითვეი და მასთან დაკავშირებული კლიენტები

სესიის სასიცოცხლო ციკლი. როდესაც მომხმარებელი სესიას იწყებს (hosting), აპლიკაცია თანმიმდევრულად ასრულებს შემდეგ ნაბიჯებს: (1) უშვებს გეითვეის პროცესს და მისგან stdout-ზე დაბეჭდილი JSON შეტყობინებით იგებს, რომელ პორტზე დაიწყო მუშაობა; (2) უშვებს cloudflared-ს და ელოდება საჯარო ბმულს; (3) HTTP მოთხოვნით გეითვეისგან ქმნის „ოთახს“ (room) და იღებს მის იდენტიფიკატორს; (4) თავად უერთდება ოთახს WebSocket-ით და მაშინვე აგზავნის დოკუმენტის სნეფშოტს, რათა მოგვიანებით შემოსული სტუმრები სწრაფად „დაეწიონ“ მიმდინარე მდგომარეობას. ამის შემდეგ ჰოსტი ორ ბმულს იღებს, - ლოკალური ქსელის და საჯაროს. სტუმრის მხარეს იგივე პროცესი ბევრად მარტივია: ბმულის ჩასმა, WebSocket კავშირი, სნეფშოტის მიღება და მუშაობის დაწყება.

სესიაში უფლებები ასიმეტრიულია: ჰოსტი ყოველთვის სრულუფლებიანია, ხოლო სტუმრები, თუ ჰოსტმა არ მიუთითა, მხოლოდ კითხვის რეჟიმში ერთვებიან (ჰოსტს შეუძლია სესიის დაწყებისას აირჩიოს, რომ სტუმრებს ჩაწერის უფლება ავტომატურად მიეცეთ). სესიის მიმდინარეობისას ჰოსტი თითოეულ სტუმარს ინდივიდუალურად ანიჭებს ან ართმევს ჩაწერის უფლებას, ამ მექანიზმის ტექნიკურ დეტალებს ქვემოთ განვიხილავთ.

Tauri ფრეიმვორკი კონცეპტუალურად

desktop აპლიკაცია აგებულია Tauri 2 ფრეიმვორკზე [6]. Tauri-ის იდეა მარტივია: აპლიკაციის ინტერფეისი იწერება ჩვეულებრივი ვებ-ტექნოლოგიებით (ჩვენს შემთხვევაში React + TypeScript) და ეშვება ოპერაციული სისტემის ჩაშენებულ webview-ში, ხოლო „ნატიური“ ლოგიკა, ფაილებთან წვდომა, ქსელი, პროცესების მართვა, იწერება Rust-ზე და ცალკე პროცესის ბირთვში ცხოვრობს. ორ სამყაროს შორის კომუნიკაცია ხდება IPC-ით: ფრონტენდი იძახებს ბექენდის „ბრძანებებს“ (commands), ბექენდი კი ფრონტენდს „ივენტებს“ (events) უგზავნის. Electron-თან შედარებით Tauri-ის ბინარები გაცილებით მცირეა (ის Chromium-ს არ აპაკეტებს) და ბექენდის ენად Rust გამოვიყენეთ. CRDT ბიბლიოთეკა სწორედ Rust-ზეა დაწერილი და აპლიკაციაში პირდაპირ, დანამატების გარეშე ჩაშენდა.

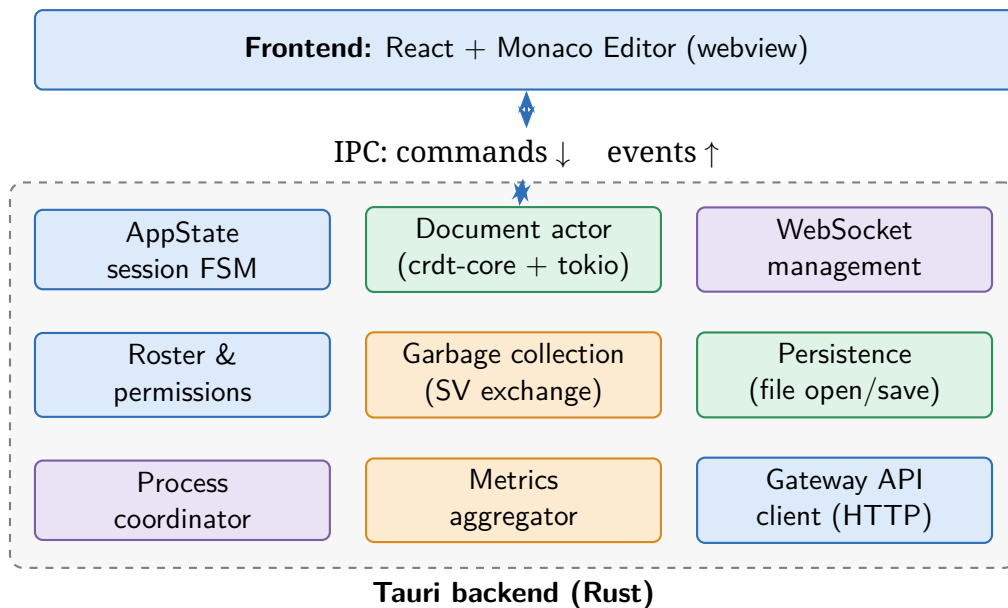
Tauri-ის კიდევ ერთი ფუნქცია, რომელსაც აქტიურად ვიყენებთ, არის sidecar ბინარები: გეითვეი და cloudflared აპლიკაციასთან ერთად იდება და აპლიკაცია მათ სასიცოცხლო ციკლს სრულად აკონტროლებს - მომხმარებელი ვერც კი ამჩნევს, რომ მის კომპიუტერზე ცალკე სერვერი გაეშვა.

ფრონტენდი და Monaco, როგორც ბაზისი

ინტერფეისის ბაზისად Monaco Editor [7] ავირჩიეთ, იგივე კომპონენტი, რომელზეც VS Code-ია აშენებული. ეს არჩევანი ბევრ რამეს გვაძლევს „უფასოდ“: სინტაქსის გაფერადება, მრავალკურსორიანი რედაქტირება, undo/redo, ძებნა და keyboard shortcuts. ჩვენი ფრონტენდი Monaco-ს გარშემოა აგებული: React მართავს აპლიკაციის შელს (გვერდითი პანელი, სესიის სტატუსი, მონაწილეთა სია, ფაილების მენიუ), ხოლო Monaco - თავად ტექსტს. ლოკალური ცვლილებები Monaco-დან ბექენდისკენ IPC ბრძანებებით მიდის, remote მონაწილეების ცვლილებები კი ბექენდიდან ივენტებით ბრუნდება.

CRDT ბიბლიოთეკა (crdt-core)

სისტემის მთავარი ნაწილია crdt-core, Rust-ზე დაწერილი დამოუკიდებელი ბიბლიოთეკა, რომელიც YATA-ს სტილის ტექსტურ CRDT-ს მოიცავს. მისი პასუხისმგებლობაა დოკუმენტის მდგომარეობა, ოპერაციების ინტეგრაცია და ბინარული (wire) ფორმატი.



დიაგრამა 2. კლიენტის (Tauri აპლიკაციის) შრეები და ქვესისტემები

ბლოკები და linked list. დოკუმენტი წარმოდგენილია ბლოკების doubly linked list-ად. ბლოკი არის ერთი ავტორის მიერ ზედიზედ აკრეფილი სიმბოლოების მონაკვეთი, რომელსაც გააჩნია:

- უნიკალური იდენტიფიკატორი $\text{BlockId} = (\text{client_id}, \text{clock})$ - ავტორის ნომერი და მისი ლოკალური მთვლელები (ლამპორტის საათის ანალოგი). n სიმბოლოიანი ბლოკი არის (c, k) -დან $(c, k+n-1)$ -მდე;
- წარმოშობის ისტორია origin_left და origin_right , რომელიც ორი ელემენტის შუაში დაიბადა ბლოკი ჩასმის მომენტში;
- შიგთავსი და წაშლის ბულებანი (deleted).

მთელი ტექსტის სიმბოლო-სიმბოლო შენახვის ნაცვლად ბლოკებით შენახვა (მიდგომა, რომელიც Yjs-იდან გადმოვიღეთ [4]) მეხსიერებასა და ქსელის ტრაფიკს მკვეთრად ამცირებს: ჩვეულებრივი ბეჭდვისას მომდევნო სიმბოლო წინა ბლოკს ეწებება/იმერჯება და ერთ ოპერაციად რჩება. ახალი ფაილის გახსნისას ტექსტი 64-სიმბოლოიან ბლოკებად იჭრება (from_text_chunked).

ბლოკის გაყოფა (splitting). რა ხდება, როდესაც ვინმე ბლოკის შუაში ჩასვამს ტექსტს ან შუიდან წაშლის? ბლოკი ორად იყოფა: მარცხენა ნახევარი ინარჩუნებს საწყის იდენტიფიკატორს, მარჯვენა კი იღებს იმავე დიაპაზონის გაგრძელებას $((c, k+\text{offset}))$. რადგან იდენტიფიკატორები სინამდვილეში დიაპაზონებზეა დამოკიდებული, ნებისმიერი BlockId

კვლავ ცალსახად პოულობს თავის ბლოკს, უბრალოდ ახლა შესაძლოა ბლოკის შეაგულში აღმოჩნდეს. ეს ინვარიანტი (ე.წ. decomposition invariance) კრიტიკულია: remote მონაწილის ოპერაცია შეიძლება ისეთ ბლოკზე განხორციელდეს, რომელიც ჩვენთან უკვე სამ ნაწილად არის გაყოფილი.

ინტეგრაცია და კონფლიქტების გადაწყვეტა. remote ბლოკის მიღებისას ალგორითმმა უნდა იპოვოს მისი ზუსტი ადგილი სიაში. თუ კონფლიქტი არ არის, ბლოკი უბრალოდ თავის origin-ებს შორის ჯდება. თუ ორმა მონაწილემ ერთსა და იმავე ადგილას ერთდროულად ჩასვა ტექსტი (ესეიგი თუ ორივე ბლოკს ერთი და იგივე origin-ები აქვს), YATA წესრიგს დეტერმინისტული წესით ადგენს, კონფლიქტური ბლოკები ისე ლაგდება, რომ (1) ყველა რეპლიკაზე ერთი და იგივე თანმიმდევრობა მიიღება და (2) ერთი ავტორის მიმდევრობითი ჩანაწერები ერთად რჩება. Rust-ის type სისტემა აქ ძალიან დაგვეხმარა: ინტეგრაციის ლოგიკა მკაცრად გამიჯნულ მოდულებშია (document/integrate.rs) და ყოველი შემთხვევა ცალკე ტესტირება დაფარული.

წაშლა, DeleteSet და state ვექტორი. CRDT-ში წაშლა განსხვავებულად მუშაობს: ბლოკის ფიზიკურად ამოღება არ შეიძლება, რადგან სხვა მონაწილის ჯერ მოუსვლელი ოპერაცია შესაძლოა სწორედ მას ეყრდნობოდეს origin-ად. ამიტომ წაშლა მხოლოდ მონიშვნაა: ბლოკი რჩება სიაში როგორც tombstone, ტექსტში კი აღარ ჩანს. წაშლები კომპაქტურად ინახება DeleteSet სტრუქტურაში, თითოეული კლიენტისთვის წაშლილი clock-დიაპაზონების სიად.

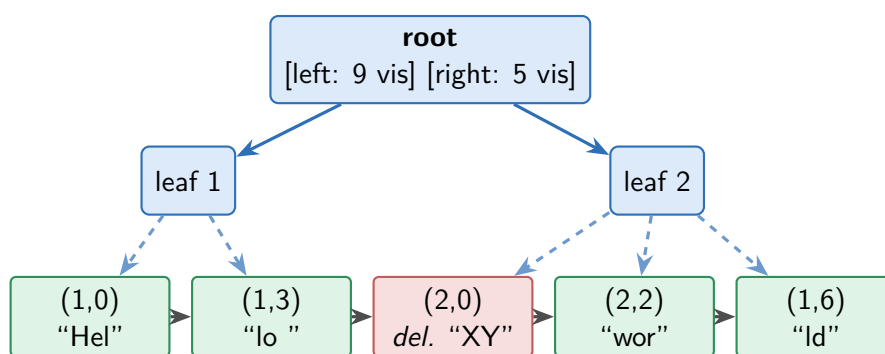
state ვექტორი ინახავს თითო კლიენტისთვის მისგან ნანახი ოპერაციების მაქსიმალურ clock-ს. ორი რეპლიკის ვექტორების შედარებით ზუსტად ჩანს, რომელ მხარეს აკლია რომელი ოპერაციები. ჩვენ ვექტორებს ორ რამეში ვიყენებთ: გვიან შემოსული სტუმრის სინქრონიზაციასა და garbage collector-ში (იხ. ოპტიმიზაციების თავი), სადაც ის განსაზღვრავს, რომელი წაშლა აქვს ყველას გარანტირებულად ნანახი.

სნეფშოტები და wire პროტოკოლი. დოკუმენტის სრული მდგომარეობა, ბლოკები, DeleteSet, state ვექტორი - სერიალიზდება Snapshot სტრუქტურად (ბინარულად, bitcode ბიბლიოთეკით). ქსელში ყოველ ფრეიმს ერთბაიტიანი პრეფიქსი აქვს, რომელიც ტიპს განსაზღვრავს: ოპერაცია (0x00), სნეფშოტი (0x01), საკონტროლო ფრეიმი (0x02), garbage collector commit (0x04), წევრობის ცვლილება (0x05), state ვექტორის

რეპორტი (0x06), უფლების ცვლილება (0x07) და მონაწილის ინფორმაცია (0x08). რადგან ამ ფორმატს ორი ენა კითხულობს (Rust კლიენტებში, Go გეითვეიში), ორივე მხარეს გვაქვს ე.წ. protocol drift ტესტები.

პოზიციური B+ ინდექსი. CRDT ოპერაციები იდენტიფიკატორებზე მუშაობს, რედაქტორი კი, სიმბოლოს პოზიციებზე („ჩასვი 42-ე პოზიციაზე“). ამ ორ სამყაროს შორის გადაყვანა linked list-ის გავლით $O(n)$ ოპერაციაა და ყოველ კლავიშზე მთელი დოკუმენტის ჩამოვლა დიდ ფაილებზე შესამჩნევად ანელებდა სისტემას. ამის გადასაჭრელად დოკუმენტს დავურთეთ აუგმენტირებული B+ ხე: ბლოკები ხის ფოთლებში დოკუმენტის თანმიმდევრობისავით აწყვია, ხოლო ყოველი შიდა node ინახავს შვილი ნოუდის (ქვე ხის) ხილული სიმბოლოების ჯამურ სიგრძეს (visible_len; წაშლილი ბლოკის ხილული სიგრძე ნულია). პოზიციით ძებნისას ხეზე დაშვებისას თითო დონეზე ვაკლებთ მარცხენა შვილების ჯამებს, $O(\log n)$; პირიქით, ბლოკიდან პოზიციის გამოთვლისას ფოთლიდან ზემოთ ავდივართ და მარცხენა ძმების ჯამებს ვაგროვებთ.

მნიშვნელოვანი დეტალი: ხე მეორეული, წარმოებული სტრუქტურაა - ჭეშმარიტების წყარო ყოველთვის doubly linked list რჩება. იმისთვის, რომ ისინი არასდროს აცდნენ ერთმანეთს, debug რეჟიმში ყოველი მუტაციის შემდეგ ეშვება ფუნქცია, რომელიც მთელ სიას ხელით ჩამოუვლის და ამოწმებს, რომ ხის პასუხები ზუსტად ემთხვევა. ხის child node size განზრახ განსხვავებულია: debug-ში მცირეა (4), რომ ტესტებში გაყოფის ლოგიკა ხშირად გავარჯიშდეს, release-ში კი დიდი რიცხვი ავიღეთ (64/32), ნაკლები სიღრმე და უკეთესი კემ-ლოკალურობა.

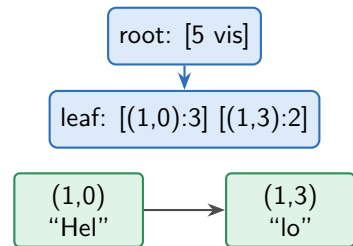


დიაგრამა 3. CRDT დოკუმენტის სტრუქტურა: ბლოკების linked list (ქვემოთ, tombstone წითლადაა) და მასზე აგებული პოზიციური B+ ინდექსი (ზემოთ, კვანძებში, ხილული სიმბოლოების ჯამები)

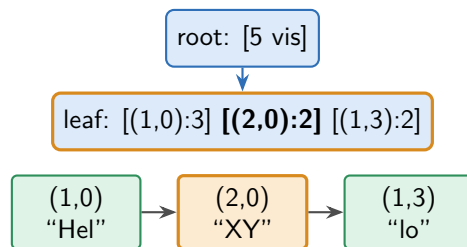
ჩასმისა და წაშლის დინამიკას ორი კადრ-კადრ დაშლილი დიაგრამა აჩვენებს (დიაგრამები 4 და 5): პირველი, თუ როგორ ჯდება ახალი ბლოკი ერთდროულად სიაშიც და ხის ფოთოლშიც და როგორ „ბუბლდება“

ხილული სიგრძის დელტა ფესვამდე; მეორე, თუ როგორ იქცევა ბლოკი tombstone-ად და როგორ სწორდება ჯამები მთელ შტოზე.

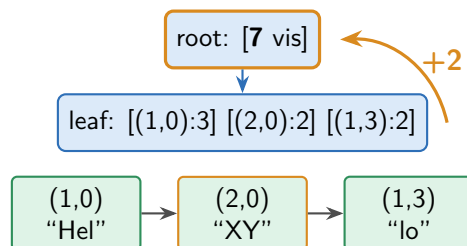
ნაბიჯი 1, საწყისი მდგომარეობა



ნაბიჯი 2, ახალი ბლოკი ჯდება სიაშიც და ფოთოლშიც



ნაბიჯი 3, დელტა (+2) ბუბლდება ფესვამდე



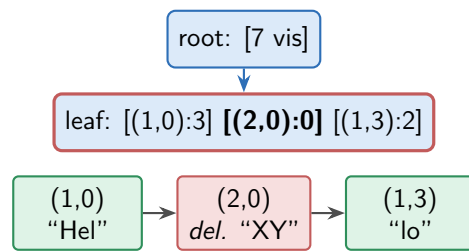
დიაგრამა 4. ბლოკის ჩასმა ნაბიჯ-ნაბიჯ: ბლოკი ერთდროულად ემატება დაკავშირებულ სიას და ხის ფოთოლს, შემდეგ კი ხილული სიგრძის დელტა ფესვამდე ვრცელდება

Tauri აპლიკაცია (desktop კლიენტი)

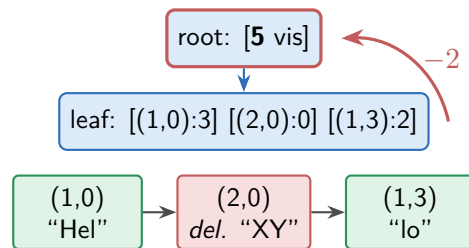
Tauri აპლიკაციის Rust ბექენდი ის ადგილია, სადაც ყველა ქვესისტემა ერთმანეთს ხვდება (იხ. დიაგრამა 2): დოკუმენტის აქტორი, WebSocket კავშირი, სესიის state machine, პროცესების მართვა, garbage collection, ფაილების შენახვა, permission სისტემა და მეტრიკები.

დოკუმენტის აქტორი. ყველაზე მნიშვნელოვანი დიზაინის გადაწყვეტილება დოკუმენტთან წვდომის ორგანიზებაა. მარტივი მიდგომა იქნებოდა `Mutex<Document>`, საზიარო ობიექტი, რომელსაც ყველა thread-ი ეპოტინება. ამის ნაცვლად აქტორის პატერნი გამოვიყენეთ: დოკუმენტს ფლობს ერთადერთი tokio task (DocActor), რომელიც უსასრულო ციკლში

ნაბიჯი 1, ბლოკი ინიშნება წაშლილად (tombstone)



ნაბიჯი 2, დელტა (-2) ბუბლდება ფესვამდე



დიაგრამა 5. ბლოკის წაშლა ნაბიჯ-ნაბიჯ: ბლოკი სიაში რჩება tombstone-ად, ხილული სიგრძე ნულდება და უარყოფითი დელტა ხის ჯამებს ასწორებს

კითხულობს შეტყობინებების არხს (mpsc). ნებისმიერი მოქმედება, ლოკალური ჩასმა, remote ოპერაცია, სნეფშოტის აღება, ფაილში ჩასაწერი ტექსტის წაკითხვა, არხში იგზავნება DocOp შეტყობინებად, პასუხი კი ერთჯერადი (oneshot) არხით ბრუნდება. ამ მიდგომით lock-ები და deadlock-ის რისკი ქრება, ოპერაციები ბუნებრივად სერიალიზდება (რაც CRDT-ისთვის ისედაც აუცილებელია), ხოლო ახალი ოპერაციის დამატება მხოლოდ ახალი DocOp ვარიანტის დაწერას ნიშნავს.

IPC, ბექენდიდან ფრონტენდამდე. როდესაც აქტორი remote ცვლილებას აინტეგრირებს, ფრონტენდს Tauri ივენთს უგზავნის: crdt://remote-change (პოზიცია და შიგთავსი), crdt://snapshot-applied (დოკუმენტი მთლიანად შეიცვალა, მაგ. სესიაში შესვლისას) ან crdt://document-reset. საპირისპირო მიმართულებით ფრონტენდი #[tauri::command] ბრძანებებს იძახებს: insert, delete, replace, ფაილის ოპერაციები, სესიის მართვა და ა.შ. მნიშვნელოვანი დეტალი: webview-სა და ბექენდს შორის შეტყობინებები ასინქრონულია, ამიტომ შესაძლებელია, რომ მომხმარებელმა პოზიციაზე დააჭიროს მაშინ, როცა ბექენდში უკვე ჩამოვიდა (მაგრამ რედაქტორში ჯერ არ ასახულა) სხვისი ცვლილება. ამის გამო ყოველი ლოკალური ბრძანება თან ატარებს base_seq-ს, ბოლო remote ცვლილების ნომერს, რომელიც რედაქტორს ჰქონდა ნანახი ბრძანების გაგზავნისას. აქტორი ინახავს ბოლო remote ოპერაციების მცირე ლოგს და შემოსულ პოზიციას მათ მიმართ

გადათვლის. ამ ხერხმა სინქრონიზაციის რთულად დასაჭერი ბაგები აღმოფხვრა.

პროცესების მართვა. სესიის დაწყებისას `process_coordinator` უშვებს გეითვეის `sidecar`-ს, `stdout`-იდან კითხულობს `{"port": N}` შეტყობინებას, შემდეგ უშვებს `cloudflared`-ს და ლოგებიდან იჭერს გაცემულ საჯარო ბმულს. გეითვეისთან ავტორიზაციისთვის აპლიკაცია ყოველ სესიაზე შემთხვევით `bearer` ტოკენს აგენერირებს და პროცესს `environment` ცვლადით გადასცემს, ამ ტოკენით ხდება ოთახის შექმნა და სესიის დასრულება. ფანჯრის დახურვისას აპლიკაცია სინქრონულად შლის ოთახს და კლავს ორივე `sidecar`-ს, რომ მომხმარებელს „ობოლი“ სერვერი არ დარჩეს ფონურად გაშვებული.

ასინქრონულობა და WebSocket მართვა. მთელი ბექენდი `tokio`-ს ასინქრონულ `runtime`-ზე დგას. `WebSocket` კავშირს სამი დამოუკიდებელი `tokio task` ემსახურება: `write_loop` (გამავალი ფრეიმების რიგი), `receive_loop` (შემომავალი ბაიტების მიღება და ტიპის ამოცნობა) და `process_loop` (მიღებული ოპერაციების გადაცემა დოკუმენტის აქტორისთვის და სხვა ქვესისტემებისთვის). ამ გამიჯვნით ნელი ქსელი ვერასდროს ბლოკავს დოკუმენტის დამუშავებას, ხოლო კავშირის გაწყვეტისას საკმარისია სამივე `task`-ის გაუქმება ერთ ადგილას (`disconnect handler`).

სესიის state machine. სესიის მდგომარეობა მკაცრი სასრული ავტომატია (FSM): `Undecided` → `Starting` → `Host/Guest` → `Undecided`. ყველა გადასვლა ვალიდირდება და უკანონო გადასვლა შეცდომას აბრუნებს. სესიის წამოწყება მრავალსაფეხურიანი ასინქრონული პროცესია და ნებისმიერ საფეხურზე შეიძლება ჩავარდეს, ამიტომ `Starting` მდგომარეობას `RAII guard` იცავს: თუ ფუნქცია შეცდომით დასრულდა, მდგომარეობა ავტომატურად ბრუნდება `Undecided`-ზე. ასე გამოვრიცხეთ „გაჭედილი“ შუალედური მდგომარეობები ხელით `rollback`-ების წერის გარეშე.

Permission მართვა. ჩაწერის უფლებების ავტორიტეტი გეითვეია: ის პირველ შემოერთებულს ჰოსტად თვლის, ყოველ კლიენტს `write` დროშას უნახავს და `read-only` კლიენტის ოპერაციებს უბრალოდ აგდებს. ჰოსტი უფლების შეცვლას სპეციალური ფრეიმით (`0x07`) ითხოვს, გეითვეი ამოწმებს, რომ მოთხოვნა ნამდვილად ჰოსტისგან მოდის, ცვლის დროშას და იმავე ფრეიმს ყველას უგზავნის. კლიენტის მხარეს `roster`

მოდული აწარმოებს მონაწილეთა სიას (სახელები, ჰოსტობა, უფლებები), ხოლო სტუმრის საკუთარი უფლება `AppRole::Guest`-შივე ინახება და `insert/delete/replace` ბრძანებებს კეტავს. ინტერფეისში ეს Monaco-ს `readOnly` რეჟიმად აისახება. აქ ერთი ხრიკი დაგვჭირდა: Monaco `readOnly` რეჟიმში პროგრამულ ცვლილებებსაც ჩუმად აგდებს, ამიტომ `remote` ცვლილების ჩასმისას დროშას წამიერად ვხსნიტ და ისევ ვაბრუნებთ.

ფაილების შენახვა და გახსნა. დოკუმენტი ორ ფორმატში ინახება, გაფართოების მიხედვით: ჩვეულებრივი ტექსტი (ნებისმიერი გაფართოება) ან საკუთარი `.pcdoc` ფორმატი, PCDC magic ბაიტები, ვერსია და ბინარულად სერიალიზებული სრული CRDT სნეფშოტი, ისტორიითა და იდენტიფიკატორებით. გახსნისას ფორმატს ვცნობთ არა გაფართოებით, არამედ magic ბაიტებით. ჩაწერა ყოველთვის ატომურია, ჯერ დროებით ფაილში, შემდეგ გადარქმევით, რომ შენახვისას შეწყვეტამ ნახევრად ჩაწერილი ფაილი არ დაგვიტოვოს. ცალკე პრობლემა აღმოჩნდა line ending-ები: Windows-ის ფაილები CRLF-ს იყენებს, Monaco და CRDT კი LF-ს ელოდება. გახსნისას ტექსტს LF-ზე ვანორმალიზებთ, ვიმახსოვრებთ, იყო თუ არა ფაილი CRLF-იანი, და შენახვისას ისევ ვაბრუნებთ, ასე Windows-ის ფაილი ბაიტ-ბაიტ იდენტური ბრუნდება და კროსპლატფორმულ სესიაშიც პოზიციები აღარ „ცურავს“. ბოლოს, აპლიკაცია ბოლო გახსნილი ფაილების სიასაც (MRU) აწარმოებს სწრაფი წვდომისთვის.

მეტრიკების სისტემა. დიაგნოსტიკისთვის აპლიკაციას ჩაშენებული მეტრიკების პანელი აქვს. გეითვეი ატომურ counter-ებს აწარმოებს, აქტიური ოთახები და კლიენტები, გადაგზავნილი ფრეიმების რაოდენობა და მოცულობა, სნეფშოტ-სინქრონიზაციების წარმატება/ჩავარდნა, და მათ HTTP ენდპოინტზე აქვეყნებს. ჰოსტის აპლიკაციაში ცალკე `tokio task (process_metrics_aggregator)` პერიოდულად კითხულობს გეითვეის ამ ენდპოინტს და `cloudflared`-ის მეტრიკებს, აერთიანებს მათ და ივენით ფრონტენდს აწვდის, სადაც მომხმარებელი მათ პანელში ხედავს. ასე მაშინვე ჩანს, გადის თუ არა ტრაფიკი და ვინ არის შეერთებული.

ფრონტენდი

ფრონტენდი React + TypeScript აპლიკაციაა (აწყობილი Vite-ით), რომლის ცენტრშიც Monaco რედაქტორი დგას. მისი მთავარი ამოცანა ორმხრივი ნაკადის გამართული მუშაობაა: ლოკალური keystrokes ბექენდისკენ, remote ცვლილებები - რედაქტორისკენ.

ლოკალური ცვლილებების გაგზავნა. Monaco-ს ყოველი ცვლილება (`onDidChangeModelContent`) ითარგმნება IPC ბრძანებად: `insert`, `delete` ან `replace`, პოზიციით და შიგთავსით. იმისთვის, რომ სწრაფი ბეჭდვისას ბრძანებებმა ერთმანეთს არ გადაუსწრონ, ისინი ერთ ჯაჭვზე ეწყობა (`opQueue`-ს მეშვეობით ყოველი შემდეგი ბრძანება წინას დასრულებას ელოდება). თითო ბრძანებას თან მიჰყვება `baseSeq`, ბოლო `remote` ცვლილების ნომერი, რომელიც რედაქტორს გაგზავნის მომენტში ჰქონდა ასახული, რომლითაც ბექენდი პოზიციას გადათვლის როცა საჭიროა.

remote ცვლილებების მიღება. ბექენდის `crdt://remote-change` ივენთს ისმენს `remoteChangeListener`, რომელიც ცვლილებას Monaco-ში პროგრამულად ჩასვამს. აქ ორი ხაფანგი გვხვდება: ჯერ ერთი, პროგრამული ჩასმაც `onDidChangeModelContent`-ს ააქტიურებს, რაც ცვლილებას უკან, ბექენდისკენ გააბრუნებდა და უსასრულო ციკლს გააჩენდა, ამას ვკეტავთ `isApplyingRemote` დროშით, რომელიც ჩასმის განმავლობაში ლოკალურ დამმუშავებელს თიშავს. მეორე, წაკითხვის რეჟიმში მყოფ რედაქტორში Monaco პროგრამულ ცვლილებებსაც უგულებელყოფს, ამიტომ `remote` ცვლილების ჩასმისას `readOnly` დროშა წამიერად იხსნება. სნეფშოტის მიღებისას (მაგ. სესიაში შესვლისას) ცალკე მსმენელი მთელ დოკუმენტს ანაცვლებს.

ინტერფეისის აგებულება. ვიზუალურად აპლიკაცია VS Code-ის სტილის გარსია: მარცხნივ ვიწრო `side rail`, რომელიც გვერდით პანელს ხსნის სამი სექციით - ფაილები (გახსნა/შენახვა/ბოლო ფაილები), თანამშრომლობა (სესიის წამოწყება/შეერთება/დატოვება) და პერსონალური პარამეტრები (სახელი, თემა, ფონტის ზომა). ზედა ზოლში სესიის სტატუსი, მონაწილეთა ავატარები (თითოეულს დეტერმინისტულად გამოთვლილი ფერი აქვს), მოსაწვევი ბმულების `popover` და მენიუა. ინტერფეისის მდგომარეობა React ჰუკებშია ორგანიზებული (`useRoomState`, `useSessionEvents`, `useWritePermission`, `useTheme` და ა.შ.), რომლებიც ბექენდის ივენთებს უსმენენ და კომპონენტებს უბრალო `props`-ებით კვებავენ.

ფრონტენდის ვიზუალური დიზაინის იტერაციებში (ფერთა პალიტრა, კომპონენტების განლაგება, თემები) დამხმარე ინსტრუმენტად ვიყენებდით Claude-ს [9]; საბოლოო დიზაინის გადაწყვეტილებები და მათი იმპლემენტაცია გუნდის წევრებისაა.

ფრონტენდს აქვს საკუთარი ტესტებიც (`vitest`): სინქრონიზაციის კრიტიკული დამხმარე ლოგიკა, შებრუნებული რედაქტირებების გამოთვლა და `baseSeq`-ის აღრიცხვა, ავტომატურად მოწმდება, ხოლო ტიპიზაცია და ლინტერი CI-ში ეშვება.

Gateway (Go relay სერვერი)

Gateway dumb relay ტიპის სერვერია: ის დოკუმენტის შესახებ არაფერს ინახავს, არც სნეფშოტს, არც ოპერაციების ისტორიას არც CRDT დოკუმენტს. მისი საქმეა ოპერაციების კლიენტებთან სწრაფად გადაგზავნა.

მისი ძირითადი ცნებებია:

- **Hub**, ოთახების რეესტრი: ქმნის ოთახებს (POST /rooms), პოულობს მათ იდენტიფიკატორით და შლის სესიის ბოლოს;
- **Room**, ერთი სესიის სივრცე: წევრების სია და გადაგზავნის ლოგიკა. მიღებული ფრეიმი ეგზავნება ოთახის ყველა წევრს გამგზავნის გარდა;
- **Client**, Room-ის წევრი კლიენტის სტრუქტურა.

მიუხედავად dumb როლისა, გეითვეი სერვერს რამდენიმე აქტიური მოვალეობა მაინც აქვს. ახალი წევრის შემოსვლისას ის ჰოსტს სნეფშოტის მოთხოვნის საკონტროლო ფრეიმს უგზავნის და ჰოსტის პასუხს მხოლოდ ახალ წევრს გადასცემს, ასე გვიან შემოსული სტუმარი სწრაფად იღებს დოკუმენტის მიმდინარე მდგომარეობას, ისე, რომ სერვერს დოკუმენტის შენახვა არ სჭირდება. წევრის ყოველ შემოსვლა-გასვლაზე ოთახი ყველა Peer-ს ატყობინებს და ახალ წევრს, არსებული წევრებზე სრულ ინფორმაციას გადასცემს. გეითვეი ასევე ინახავს Permission data-ს კლიენტებზე და კლიენტებისგან შემოსული data-ზე ახდენს შესაბამის ვალიდაციებს: არ აკეთებს ისეთი ფრეიმების დაგზავნას, რომლის გამომგზავნსაც write პრივილეგია არ აქვს.

უსაფრთხოებისა და მდგრადობის მხრივ: ყველა HTTP ენდპოინტი (გარდა /health და /ws) მოითხოვს bearer ტოკენს, რომელიც მხოლოდ ჰოსტის აპლიკაციამ იცის; HTTP endpoint-ები დაცულია IP Rate Limiter-ით.

სისტემის ოპტიმიზაცია

პროექტის მეორე ნახევარში ორი დიდი ოპტიმიზაცია განვახორციელეთ: პოზიციური B+ ინდექსი და garbage collection სისტემა. ორივე იმ პრობლემებს პასუხობს, რომლებიც ყველა სერიოზულ CRDT იმპლემენტაციას აწუხებს.

პოზიციური B+ ინდექსი რიცხვებში. ინდექსის აგებულება CRDT-ის თავში აღწერეთ; აქ მის ეფექტზე ვისაუბრებთ. ოპტიმიზაციამდე ყოველი keystroke დოკუმენტის linked list-ის თავიდან ჩამოვლას მოითხოვდა,

$O(n)$ ორივე მიმართულებით (პოზიციიდან ბლოკამდე და ბლოკიდან პოზიციამდე). მცირე ფაილებზე ეს შეუმჩნეველია, მაგრამ ღირებულება ბლოკების რაოდენობასთან ერთად წრფივად იზრდება და ის ყოველ keystroke-ზე და ყოველ remote ოპერაციაზე გადაიხდება. ჩვენი გაზომვებით (debug ბილდი, სადაც ხის branching factor-ი განზრახ მცირეა):

ბლოკების რაოდენობა	სიის ჩამოვლა ($O(n)$)	B+ ხე ($O(\log n)$)	აჩქარება
~200	~127 μs	~14 μs	9×
1 000	~620 μs	~16 μs	~ 39×
10 000	~6 200 μs	~18 μs	~ 340×

ცხრილი 1. პოზიციის ძებნის დრო ინდექსამდე და ინდექსის შემდეგ

მთავარი დაკვირვება ისაა, რომ ხის სვეტი თითქმის მუდმივია. Release ბილდში (branching 64/32, სრული ოპტიმიზაციები) თითო ძებნა 1 μs -ზე ნაკლებია დოკუმენტის ზომის მიუხედავად, ანუ რედაქტორის frame budget-ს ინდექსი ვერასდროს ემუქრება. ეს ის მიზეზია, რის გამოც დიდი CRDT იმპლემენტაციები (Yjs, diamond-types [5]) მსგავს ინდექსებს აუცილებლობად თვლიან.

Garbage collection, რატომ არის საჭირო. როგორც აღვნიშნეთ, CRDT-ში წაშლა მხოლოდ მონიშვნაა და წაშლილი ბლოკები tombstone-ებად რჩება. აქტიური რედაქტირებისას ეს სწრაფად გროვდება: დიდხანს მიმდინარე სესიაში დოკუმენტის მეხსიერების დიდი ნაწილი შესაძლოა დიდი ხნის წაშლილმა ტექსტმა დაიკავოს, რომელიც სწავსწრეებთან ერთად ქსელშიც მოგზაურობს. tombstone-ის უბრალოდ წაშლა კი საშიშია: თუ რომელიმე მონაწილემ ის ჯერ არ ნახა, ან თუ დაგვიანებული ოპერაცია მას origin-ად ეყრდნობა, ნაადრევი წაშლა რეპლიკების divergence-ს გამოიწვევს.

ჩვენი მიდგომის თავისებურება. აქ ვისარგებლეთ ჩვენი არქიტექტურის ერთი თვისებით, რომელიც „სუფთა“ peer-to-peer სისტემებს არ აქვთ: ჩვენს სესიას ჰყავს გამორჩეული კვანძი - ჰოსტი, რომელიც ყოველთვის ონლაინაა (სესია მის გარეშე ვერ არსებობს) და რომელსაც გეითვეი წევრობის ზუსტ სურათს აწვდის. ეს ჰოსტს ბუნებრივ კოორდინატორად აქცევს, ისე, რომ დოკუმენტის მონაცემები მაინც სრულად დეცენტრალიზებული რჩება. სქემა ასე მუშაობს:

1. ყოველი სტუმარი პერიოდულად (debounce-ით, რომ ქსელი არ გადაიტვირთოს) აქვეყნებს თავის state ვექტორს: „აი, რამდენი მაქვს ნანახი თითოეული ავტორისგან“;

2. ჰოსტი აგროვებს ყველა წევრის ვექტორს (გეითვის membership ფრეიმები ეუბნება, ვინ არის ამჟამად ოთახში) და ითვლის მათ მინიმუმს, ე.წ. floor-ს: ოპერაციების იმ ზღვარს, რომელიც ყველას გარანტირებულად აქვს ნანახი;
3. ამ floor-ს ჰოსტი კვეთს საკუთარ DeleteSet-თან და იღებს „დადასტურებული“ წაშლების სიმრავლეს, წაშლებს, რომლებიც ყველამ ნახა;
4. დადასტურებული დიაპაზონები gc-commit ფრეიმად ეგზავნება ყველა მონაწილეს (და ჰოსტისვე დოკუმენტს), რომლებიც მას ერთნაირად ასრულებენ.

gc-commit-ის შესრულება განზრახ კონსერვატიულია: ბლოკები სტრუქტურულად არ იშლება, მხოლოდ მათი ტექსტური შიგთავსი იცლება და სნეფშოტისთვის განკუთვნილი წაშლის სიმრავლეები იკვეცება. ბლოკის იდენტიფიკატორი და ადგილი სიაში უცვლელი რჩება, ამიტომ დაგვიანებული ოპერაცია, რომელიც ამ ბლოკს origin-ად იყენებს, ისევ ზუსტად ინტეგრირდება. სწორედ ეს ხდის ჩვენს garbage collection-ს უსაფრთხოს. ბლოკების სტრუქტურული წაშლა ინტეგრაციის ალგორითმის ინვარიანტებს არღვევს და მეხსიერების მოგება რისკად არ ღირს, მთავარი მოგება (ტექსტის გათავისუფლება და სნეფშოტების შემცირება) ისედაც მიიღწევა.

პრობლემები, რომლებსაც გზაში გადავაწყდით. აქვე შევაჯამებთ განვითარებისას წამოჭრილ მნიშვნელოვან პრობლემებსა და მათ გადაწყვეტებს:

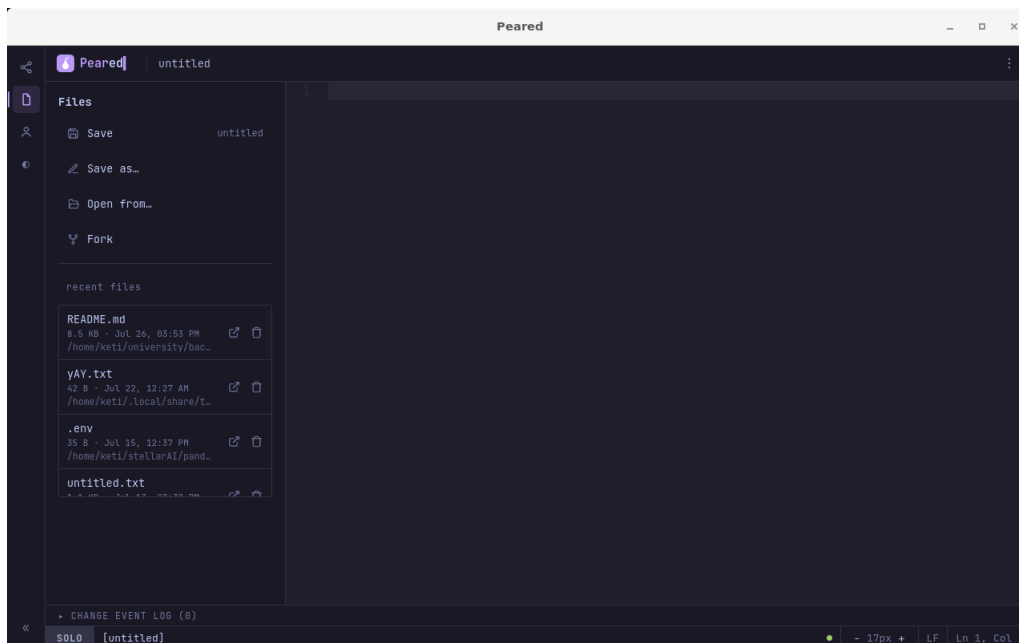
- **პოზიციების „ცურვა“** webview-სა და ბექენდს შორის - ასინქრონული IPC-ის გამო ლოკალური ბრძანება შესაძლოა მოძველებულ პოზიციას ატარებდეს. გადაწყვეტა: baseSeq + ბექენდში remote ოპერაციების მცირე ლოგის მიმართ პოზიციის გადათვლა.
- **Windows-ის CRLF დესინქრონიზაცია**, CRLF-იანი ფაილის გახსნისას პოზიციები პლატფორმებს შორის აღარ ემთხვეოდა და ხაზის შუაში რედაქტირება ცდებოდა. გადაწყვეტა: LF-ზე ნორმალიზაცია გახსნისას და CRLF-ის აღდგენა შენახვისას.
- **Monaco-ს readOnly ხაფანგი**, read-only რეჟიმში Monaco პროგრამულ რედაქტირებებსაც აგდებდა და სტუმარი remote ცვლილებებს ვეღარ იღებდა. გადაწყვეტა: დროშის წამიერი მოხსნა remote ცვლილების ჩასმისას.

- **სნეფშოტ-სინქრონიზაციის race**, თავდაპირველად ჰოსტი სნეფშოტს მხოლოდ შეერთებისას აგზავნიდა და გვიან შემოსული სტუმარი შესაძლოა მოძველებულ მდგომარეობას იღებდა. გადაწყვეტა: request- response სქემა, გეითვეი ყოველ ახალ წევრზე ჰოსტს ახალ სნეფშოტს სთხოვს.
- **დიდი ჩასმების დანაწევრება**, გიგანტური ერთიანი ბლოკები (მაგ. მთელი ფაილის ჩასმა) ინდექსსა და ქსელს ტვირთავდა. გადაწყვეტა: ტექსტის 64-სიმბოლოიან ბლოკებად დაჭრა გახსნისას.
- **ინდექსისა და სიის აცდენა**, B+ ხის დანერგვისას ყველაზე სახიფათო რისკი ორი სტრუქტურის ერთმანეთისგან აცდენა იყო. გადაწყვეტა: debug რეჟიმის შემოწმება, რომელიც ყოველი მუტაციის შემდეგ სრულ შედარებას აკეთებს, პლუს ხის შიდა ინვარიანტების ცალკე ვალიდატორი.

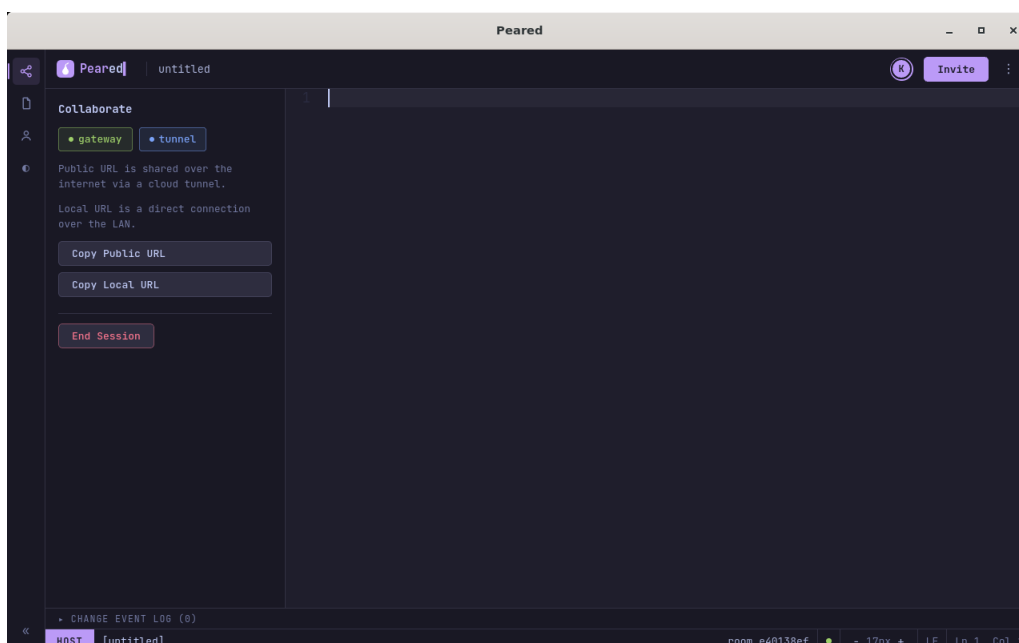
მიღებული შედეგი და შეფასება

პროექტის საბოლოო შედეგი არის მომუშავე, კროსპლატფორმული desktop აპლიკაცია, რომელიც Linux-ზე, Windows-სა და macOS-ზე ეშვება. ტიპური workflow ასე გამოიყურება: პირველი გაშვებისას მომხმარებელი სახელს ირჩევს (იმახსოვრება შემდეგი გაშვებებისთვის), შემდეგ ხსნის ან ქმნის დოკუმენტს და გვერდითი პანელიდან იწყებს სესიას (ფაილების სექცია, იხ. სურათი 1). აპლიკაცია რამდენიმე წამში აბრუნებს ორ მოსაწვევ ბმულს, ლოკალური ქსელისა და საჯაროს (იხ. სურათი 2). სტუმრები საკუთარ აპლიკაციაში ბმულს ჩასვამენ და მაშინვე ხედავენ დოკუმენტის მიმდინარე მდგომარეობას; ყველა შემდგომი ცვლილება ყველასთან რეალურ დროში აისახება. ჰოსტი peers პანელიდან მართავს, ვის შეუძლია რედაქტირება (იხ. სურათი 3), ხოლო ნამუშევარი ნებისმიერ დროს ინახება როგორც ჩვეულებრივ ტექსტურ ფაილად, ისე .pcdoc ფორმატში (სრული ისტორიით). სესიის მდგომარეობის სადიაგნოსტიკოდ ჩაშენებულია მეტრიკების პანელი (იხ. სურათი 4).

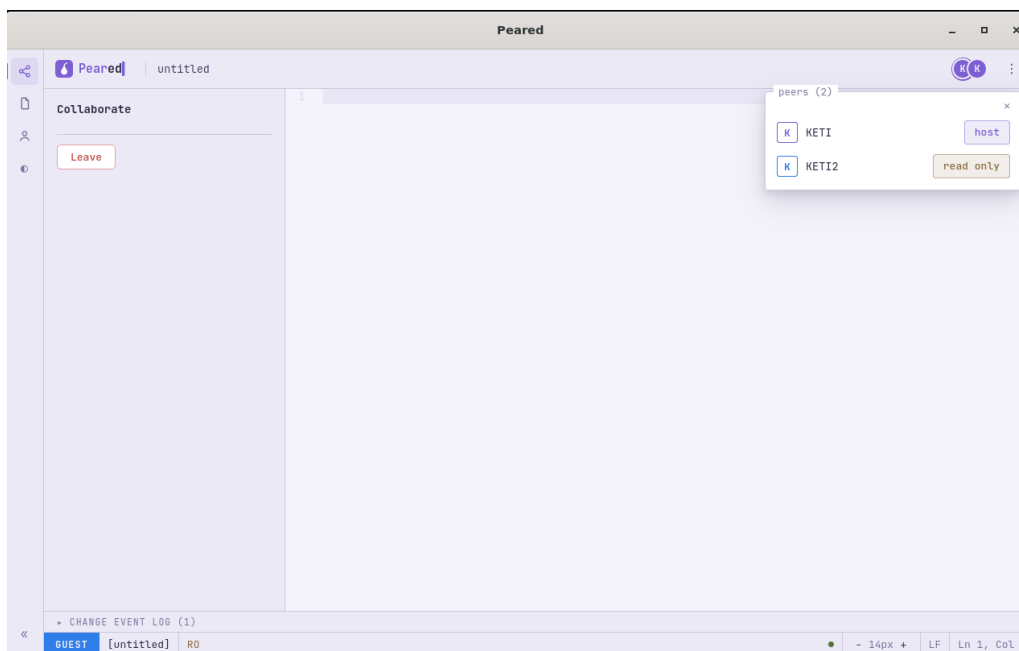
მიზნების შესრულება. შესავალში ჩამოყალიბებული ოთხივე მიზანი მიღწეულია: (1) სესიის დაწყება და ბმულით შეერთება მუშაობს ორივე ქსელურ რეჟიმში, ლოკალურ ქსელში ინტერნეტის გარეშე და გლობალურად Cloudflare Tunnel-ით; (2) პარალელური რედაქტირება უკონფლიქტოდ ერწყმის YATA CRDT-ით და მონაწილეები გარანტირებულად ერთსა და იმავე ტექსტს ხედავენ; (3) ინფრასტრუქტურის მართვა სრულად ავტომატიზებულია, მომხმარებელი



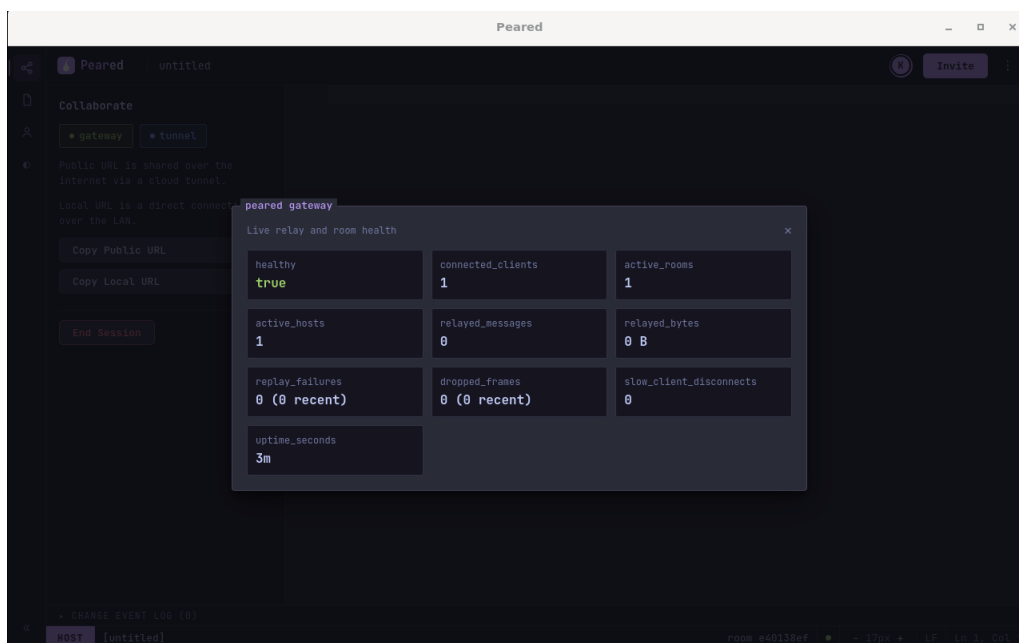
სურათი 1. ფაილების სექცია: შენახვა, გახსნა, Fork და ბოლო ფაილების სია



სურათი 2. ჰოსტის ხედი: gateway/tunnel სტატუსი, მოსაწვევი ბმულები და Invite ღილაკი (statusline-ზე ჩანს HOST როლი და room id)



სურათი 3. სტუმრის ხედი read-only რეჟიმში და peers პანელი, სადაც ჩანს ჰოსტი და read-only სტუმარი



სურათი 4. მეტრიკების პანელი: გეითვეის live სტატისტიკა (კლიენტები, ოთახები, გადაგზავნილი ფრეიმები, uptime)

სერვერს ვერც ამჩნევს; (4) ჰოსტი სესიას სრულად აკონტროლებს უფლებების ინდივიდუალური მართვით.

ხარისხის უზრუნველყოფა. სამივე კომპონენტს საკუთარი ტესტები აქვს: CRDT ბიბლიოთეკაში ინტეგრაციის, splitting-ის, წაშლისა და ინდექსის ტესტები (მათ შორის კონფლიქტური სცენარები), გეითვეიში ოთახის, გადაგზავნისა და permission ტესტები, ფრონტენდზე სინქრონიზაციის ლოგიკის ტესტები. ბინარულ პროტოკოლს ორივე მხარეს protocol drift ტესტები იცავს. CI (GitHub Actions) ყოველ ცვლილებაზე უშვებს ფორმატირების, ლინტერების, ტესტებისა და უსაფრთხოების შემოწმებებს (cargo audit, npm audit, govulncheck).

დარჩენილი შეზღუდვები. აღსანიშნავია ისიც, რაც სისტემას ჯერ არ შეუძლია: სესია ერთ დოკუმენტზეა შემოსაზღვრული (მრავალფაილიანი პროექტების მხარდაჭერა არ არის), მონაწილეთა კურსორები ერთმანეთისთვის ჯერ არ ჩანს, ხოლო ჰოსტის გათიშვა სესიას ასრულებს. ეს პუნქტები სამომავლო განვითარების ბუნებრივი კანდიდატებია (იხ. დასკვნა).

დასკვნა

Peared-ის შექმნამ ერთ პროექტში მოაქცია ის, რასაც ჩვეულებრივ რამდენიმე სხვადასხვა კურსი ფარავს: დისტრიბუციული სისტემები (CRDT, State Vector Exchange სისტემა, Garbage collection სისტემა), მონაცემთა სტრუქტურები (აუგმენტირებული B+ ხე), სისტემური პროგრამირება Rust-ზე, ქსელური პროგრამირება Go-ზე და თანამედროვე დესკტოპ/ვებ ინტერფეისები. საბოლოო პროდუქტი, ერთობლივი რედაქტორი, რომელიც ცენტრალური სერვისის გარეშე მუშაობს როგორც ლოკალურ, ისე გლობალურ ქსელში, თავიდანვე დასახულ მიზანს პასუხობს და რეალურ სცენარში (აუდიტორია არასტაბილური ინტერნეტით) ნამდვილად გამოსადეგია.

ტექნიკური ცოდნის მიღმა პროექტმა რამდენიმე უფრო ზოგადი გაკვეთილი გვასწავლა. პირველი, კორექტულობის ინფრასტრუქტურის ღირებულება: debug- ორაკულმა, protocol drift ტესტებმა და მკაცრმა CI-მ განაწილებული სისტემის ყველაზე საშიში კლასის შეცდომები (რეპლიკების უხმო დაშორება) განვითარების ეტაპზე დაიჭირა, სანამ ისინი „იშვიათ, გაუმეორებელ ბაგებად“ იქცეოდნენ. მეორე, აბსტრაქციის საზღვრების სწორად გავლება: ის, რომ crdt-core არაფერს იცის ქსელზე, გეითვეი არაფერს იცის დოკუმენტზე, ხოლო ფრონტენდი არაფერს იცის CRDT-ზე, სამივე კომპონენტის დამოუკიდებლად ტესტირებას და პარალელურად განვითარებას აძლევდა საშუალებას. მესამე, გუნდური მუშაობის დისციპლინა: სამი ადამიანის ნამუშევრის ყოველდღიური შერწყმა pull request-ებით, კოდის განხილვით და ავტომატური შემოწმებებით თავად პროექტის ისეთივე ნაწილი აღმოჩნდა, როგორც ალგორითმები.

სამომავლოდ პროექტს რამდენიმე მკაფიო მიმართულება აქვს: მონაწილეთა კურსორების ჩვენება (presence), მრავალფაილიანი პროექტების მხარდაჭერა, ჰოსტის როლის გადაცემა სესიის შეუწყვეტლად და end-to-end შიფრაცია. ასევე საინტერესოა .pcdoc ფორმატში ისედაც შენახული ისტორიის გამოყენება „დროში გადახვევის“ ფუნქციისთვის.

გამოყენებული ლიტერატურა

- [1] P. Nicolaescu, K. Jahns, M. Derntl, and R. Klamma, “Near Real-Time Peer-to-Peer Shared Editing on Extensible Data Types,” in Proceedings of the 19th International Conference on Supporting Group Work (GROUP ’16), Sanibel Island, FL, USA, 2016, pp. 39–49.
- [2] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free Replicated Data Types,” in Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS ’11), Grenoble, France, 2011, pp. 386–400.
- [3] C. A. Ellis and S. J. Gibbs, “Concurrency control in groupware systems,” in Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data, Portland, OR, USA, 1989, pp. 399–407.
- [4] K. Jahns, “Yjs Documentation.” [Online]. Available: <https://docs.yjs.dev/>
- [5] J. Gentle, “diamond-types: The world’s fastest CRDT.” [Online]. Available: <https://github.com/josephg/diamond-types>
- [6] Tauri Working Group, “Tauri 2.0 Documentation.” [Online]. Available: <https://v2.tauri.app/>
- [7] Microsoft, “Monaco Editor.” [Online]. Available: <https://microsoft.github.io/monaco-editor/>
- [8] Cloudflare, Inc., “Cloudflare Tunnel Documentation.” [Online]. Available: <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>
- [9] Anthropic, “Claude.” [Online]. Available: <https://claude.com/>
(გამოყენებულია ფრონტენდის ვიზუალური დიზაინის იტერაციებში დამხმარე ინსტრუმენტად)

დანართები

- პროექტის GitHub ბმული: <https://github.com/CarlTeapot/Peared>